CORRIGÉ · MISSION 1

Le premier panier

Dans missions/01_panier.R, créer d'abord les six objets d'entrée, puis calculer à partir de ces noms. Une ligne commençant par # est un commentaire.

```
prix_cafe <- 3
quantite_cafe <- 2
prix_the <- 5
quantite_the <- 1
prix_sandwich <- 8
quantite_sandwich <- 1
total_panier <- prix_cafe * quantite_cafe +
  prix_the * quantite_the +
  prix_sandwich * quantite_sandwich
cout_panier <- quantite_cafe * 1 +
  quantite_the * 2 + quantite_sandwich * 3
marge_panier <- total_panier - cout_panier
```

RÉSULTAT ET UNITÉ

19 TND de ventes, 7 TND de coût, 12 TND de marge. Afficher les objets en écrivant leurs noms dans le script.

Pour l'essai avec un café supplémentaire : 22 TND de ventes, 8 TND de coût et 14 TND de marge. La preuve de mission reste le panier initial. Garder cet essai dans un autre objet pour préserver les entrées demandées.

CORRIGÉ · MISSION 2

Les quatre jours

```
ventes_journalieres <- c(19, 27, 33, 21)
total_semaine <- sum(ventes_journalieres)
moyenne_jour <- mean(ventes_journalieres)
print(length(ventes_journalieres))
print(total_semaine)
print(moyenne_jour)
```

RÉSULTAT ET UNITÉ

4 jours ; 100 TND au total ; 25 TND par jour. Malgré le nom semaine, le carnet couvre seulement quatre jours.

La moyenne est 100 / 4. Elle exprime un montant par jour ; le total exprime le montant de toute la période. L'essai à cinq jours donne 130 TND et 26 TND par jour, dans un autre objet.

Preuve : missions/02_semaine.R enregistré et résultats dans la console. Aucun export n'est nécessaire. Relancer puis exécuter source("checks/02_semaine.R").

CORRIGÉ · MISSION 3

Lire une vente

```
commandes <- read.csv("data/orders_raw.csv")
print(nrow(commandes))
premiere_commande <- commandes[1, ]
produits <- unique(commandes$produit)
print(premiere_commande)
print(produits)
```

RÉSULTAT ET UNITÉ

14 lignes brutes. Première commande : O001, Cafe, quantité 2, prix unitaire 3 TND, coût unitaire 1 TND. Trois produits distincts : Cafe, The et Sandwich.

$ choisit une colonne. [1,] choisit toute la première ligne. unique() garde chaque valeur différente une fois. Une ligne décrit une commande d'un produit ; elle peut contenir plusieurs unités.

Essai analogue : commandes[3, "produit"] renvoie Sandwich, et commandes[3, "quantite"] renvoie 1. Ne pas modifier le fichier brut.

CORRIGÉ · MISSION 4

Le fichier propre

Comparer après votre essai. Les solutions complètes du dossier reprennent aussi les imports et les exports nécessaires pour relancer chaque mission.

```
commandes <- read.csv("data/orders_raw.csv")
sans_doublon <- unique(commandes)
a_completer <- sans_doublon[
  is.na(sans_doublon$quantite), ]
propres <- sans_doublon[
  !is.na(sans_doublon$quantite), ]
print(nrow(propres))
```

RÉSULTAT ET UNITÉ

13 lignes après retrait de la copie ; O013 mise à part ; 12 lignes conservées.

unique() compare les lignes entières. is.na() repère la quantité inconnue ; ! garde le cas contraire. La quantité absente demeure NA dans le fichier à vérifier.

VOTRE CONTRÔLE

Relancer votre propre script depuis son début. Montrer ce que vous avez changé, où le résultat est enregistré et pourquoi la réponse a du sens.

CORRIGÉ · MISSION 5

La marge par produit

Créer les colonnes chiffre_affaires, cout_total et marge comme dans le guide. Chaque calcul combine des valeurs de la même ligne. La préparation fournie importe votre export de mission 4 s'il existe, sinon la référence annoncée ; cette aide permet de commencer ici.

```
bilan_produits <- aggregate(
  cbind(chiffre_affaires, cout_total, marge) ~
    produit, data = propres, FUN = sum)
bilan_jours <- aggregate(
  cbind(chiffre_affaires, cout_total, marge) ~
    date, data = propres, FUN = sum)
total_ca <- sum(propres$chiffre_affaires)
total_marge <- sum(propres$marge)
```

Produit	Ventes / TND	Coût / TND	Marge / TND
Cafe	30	10	20
Sandwich	40	15	25
The	30	12	18

cbind() réunit les trois mesures ; ~ choisit leur groupe ; sum les additionne. Exporter bilan_produits dans outputs/05_bilan_produits.csv avec row.names = FALSE. Contrôle : 100 − 37 = 63 TND.

CORRIGÉ · MISSION 6

Les barres de marge

Comparer après votre essai. Les solutions complètes du dossier reprennent aussi les imports et les exports nécessaires pour relancer chaque mission.

```
png("outputs/06_marge_produits.png",
  width = 900, height = 600)
barplot(bilan_produits$marge,
  names.arg = bilan_produits$produit,
  main = "Marge par produit",
  ylab = "Marge (TND)", col = "steelblue")
dev.off()
```

RÉSULTAT ET UNITÉ

Trois barres de 20, 25 et 18 TND ; le sandwich est le plus haut.

png() commence l'enregistrement ; dev.off() le termine. Les noms et les hauteurs viennent du même tableau pour éviter de mélanger les produits.

VOTRE CONTRÔLE

Relancer votre propre script depuis son début. Montrer ce que vous avez changé, où le résultat est enregistré et pourquoi la réponse a du sens.

CORRIGÉ · MISSION 7

La note de décision

Comparer après votre essai. Les solutions complètes du dossier reprennent aussi les imports et les exports nécessaires pour relancer chaque mission.

```
ventes_sandwich <-
  bilan_produits$chiffre_affaires[
    bilan_produits$produit == "Sandwich"]
part_sandwich <- ventes_sandwich /
  sum(bilan_produits$chiffre_affaires) * 100
print(part_sandwich)
produit_prioritaire <- "Sandwich"
note <- c(
  "Sandwich : 40 TND, soit 40 % des ventes.",
  "Sa marge sur ingredients est 25 TND.",
  "Je propose un petit test de mise en avant.",
  "Quatre jours : verifier de nouvelles journees."
)
writeLines(note, "outputs/07_decision.txt")
write.csv(bilan_produits,
  "outputs/07_bilan_produits.csv",
  row.names = FALSE)
```

RÉSULTAT ET UNITÉ

40 % du chiffre d'affaires vient du sandwich.

La marge de 25 TND diffère des ventes de 40 TND : elle retire 15 TND d'ingrédients. La note relie les nombres à une action limitée et rappelle la courte période. Loyer, salaires et autres charges restent hors du calcul de marge.

VOTRE CONTRÔLE

Relancer votre propre script depuis son début. Montrer ce que vous avez changé, où le résultat est enregistré et pourquoi la réponse a du sens.

CORRIGÉ · MISSION 8

La recette réutilisable

Définir resumer_commandes(chemin) avec les étapes import → unique → filtre des quantités connues → calculs. Renvoyer un data.frame d'une ligne avec chiffre_affaires, cout_total, marge, quantite et n_commandes.

```
resultat_semaine_1 <- resumer_commandes(
  "data/orders_raw.csv")
resultat_semaine_2 <- resumer_commandes(
  "data/orders_week2_raw.csv")
write.csv(resultat_semaine_2,
  "outputs/08_resume_semaine_2.csv",
  row.names = FALSE)
```

Carnet	Ventes / TND	Coût / TND	Marge / TND
1 · quatre jours	100	37	63
2 · deux jours	48	18	30

Premier carnet : 21 unités, 12 commandes utilisables. Deuxième : 9 unités, 6 commandes utilisables. Le chemin change ; les règles de traitement restent les mêmes. Les périodes ont des durées différentes : ces totaux ne suffisent pas à comparer une performance.

> VOTRE CONTRÔLE
>
> Relancer la définition avant les appels. source("checks/08_automatiser.R") vérifie votre fonction sur les deux fichiers et sur un essai supplémentaire : une fonction doit calculer, pas renvoyer toujours des nombres écrits à l'avance.